Model Card

ResNet18-GPT2 Vision-to-Language Maze Solver

A Prefix-Tuning Approach for Visual Navigation Sequence Generation

Seena Mohajeran and Arshdeep Dhillon

1 Model Overview

Quick Reference

Model Type:	Vision-to-Language (Image Captioning variant)
Architecture:	ResNet18 encoder + GPT-2 decoder with prefix-tuning
Task:	Generate navigation sequences from maze images
Input:	RGB maze image (224×224 pixels)
Output:	Sequence of directional tokens (R, U)
Parameters:	12,215,488 total

1.1 Model Description

This model investigates whether visual spatial information can be effectively encoded into language model representations through prefix-tuning. The architecture combines a pretrained ResNet18 vision encoder with a small GPT-2 language decoder, connected via learned prefix embeddings that project visual features into the language model's embedding space.

The model takes a maze image as input and autoregressively generates a sequence of navigation commands (Right "R" and Up "U") that traverse from the start position (bottom-left) to the goal position (top-right). This task requires the model to:

1. Extract spatial structure from the visual input
2. Encode wall positions and valid paths
3. Generate a coherent sequence respecting maze constraints

1.2 Intended Use

Primary Use Cases:

- Research into vision-to-language information transfer
- Investigating prefix-tuning as a cross-modal bridging mechanism
- Educational demonstration of multimodal architectures
- Baseline for visual navigation tasks

Out-of-Scope Uses:

- Real-world robot navigation
- Safety-critical pathfinding applications
- General-purpose image captioning
- Mazes with different start/goal configurations

2 Architecture Details

2.1 Model Architecture

Component	Specification	Details
Vision Encoder	Base Model	ResNet18 (ImageNet pretrained)
	Output Dimension	512
	Training Status	Frozen
Feature Projection	Architecture	Linear(512 → 128)
	Activation	None
Prefix Generator	Layer 1	Linear(128 → 256) + Tanh
	Layer 2	Linear(256 → 16 × 128)
	Output Shape	[batch, 16, 128]
GPT-2 Decoder	Hidden Size	128
	Layers	2
	Attention Heads	2
	Max Positions	128
	Dropout	0.4

2.2 Tokenizer

Token ID	Token
0	`<pad>` (Padding)
1	`<s>` (Beginning of Sequence)
2	`</s>` (End of Sequence)
3	`<unk>` (Unknown)
4	`R` (Move Right)
5	`U` (Move Up)

Vocabulary Size: 6 tokens

3 Training Details

3.1 Training Data

Parameter	Value
Grid Size	7×7
Training Samples	36,950
Test Samples	9,250
Variations per Solution	50
Start Position	(0, 6) — Bottom-left
Goal Position	(6, 0) — Top-right
Random Seed	69
Image Resolution	224×224 RGB

The dataset consists of procedurally generated maze images with corresponding solution paths. Each unique maze solution is augmented with 50 visual variations to improve generalization. Mazes use a binary representation where white cells indicate valid paths and black cells indicate walls.

3.2 Training Configuration

Hyperparameter	Value
Epochs	100
Batch Size	32
Base Learning Rate	5×10^{-4}
Optimizer	AdamW
Weight Decay	0.01
LR Schedule	Cosine Annealing
Minimum LR	1×10^{-6}
Gradient Clipping	Max norm 1.0

Differentiated Learning Rates:

- ResNet features: lr/10 (frozen, effectively 0)
- Feature projection: lr
- Prefix generator: lr
- GPT-2 decoder: lr/5

3.3 Image Preprocessing

1. Resize to 224×224 pixels
2. Convert to tensor (scale to [0, 1])
3. Normalize with ImageNet statistics:
   - Mean: [0.485, 0.456, 0.406]
   - Std: [0.229, 0.224, 0.225]

4 Limitations

Limitations & Risks

1. **Grid Size Constraint:** Trained exclusively on 7×7 grids; performance on other sizes is untested and likely poor.
2. **Fixed Start/Goal:** Only supports bottom-left to top-right navigation; cannot handle arbitrary start/goal positions.
3. **Limited Action Space:** Only Right (R) and Up (U) moves; cannot handle mazes requiring Left or Down movements.
4. **Capacity Bottleneck:** The 128-dimensional hidden size and 16 prefix tokens may be insufficient for encoding complex maze structures. Prior experiments on smaller grids suggest this is a key limiting factor.
5. **Wall Collision Errors:** The majority of failures (99.3%) involve hitting walls, indicating imprecise spatial encoding.
6. **No Uncertainty Quantification:** Model provides no confidence scores or indication of prediction reliability.
7. **Single Solution Generation:** Cannot generate multiple alternative paths or rank solutions.

5 Technical Specifications

5.1 Compute Infrastructure

Component	Specification
Hardware	CUDA-compatible GPU
Framework	PyTorch
Vision Backbone	torchvision (ResNet18)
Language Model	Hugging Face Transformers (GPT2LMHeadModel)
Python Version	3.12

5.2 Model Files

File	Description
`model.py`	Model architecture definition
`tokenizer.py`	Sequence tokenizer
`dataset.py`	Dataset and DataLoader utilities
`train_utils.py`	Training loop and evaluation
`solution_validator.py`	Path validation logic
`data_utils.py`	Data loading utilities
`resnet_gpt2_prefix.pth`	Trained model checkpoint

5.3 Inference Example

```
from model import ResNetGPT2PrefixModel
from tokenizer import SimpleTokenizer

# Load model
model = ResNetGPT2PrefixModel(
    vocab_size=6,
    hidden_size=128,
    num_layers=2,
    num_attention_heads=2,
    num_prefix_tokens=16
)
model.load_state_dict(checkpoint['model_state_dict'])

# Generate solution
output = model.generate(
    images,           # [batch, 3, 224, 224]
    max_length=20,
    bos_token_id=1,
    eos_token_id=2,
    pad_token_id=0
)
```

6 Citation

If you use this model or methodology in your research, please cite:

```
@misc{resnet_gpt2_maze_solver,
  title={ResNet-GPT2 Vision-to-Language Maze Solver:
         A Prefix-Tuning Approach},
  author={[Author Name]},
  year={2025},
  note={Model Card v1.0}
}
```

This model card follows the format proposed by Mitchell et al. (2019) "Model Cards for Model Reporting"